COSC204
Computer Systems

Introduction to C

Overview

- Last Lecture
  - Calling Conventions
- This Lecture
  - C syntax
  - Source
    - K&R:
      - Brian Kernighan and Dennis Ritchie (1988), The C Programming Language. 2nd Edition. Prentice Hall. ISBN:0-13-110362-8
- Next Lecture
  - C Memory, Variables, and Pointers

History of C

- Developed at Bell Labs by Dennis Ritchie between 1972 and 1973 to construct utilities for Unix.
- C has been standardized by ANSI since 1989
  - Prior versions are known as K&R C
  - ANSI C (C89), ISO ratified (C90)
  - C99
    - Inline functions, long long int, // comments, etc.
  - C11
    - Thread local storage, anonymous structs, etc.
  - C17 (also known as C18)
    - Fixed “defects” in C11
  - C23
    - Added to the standard library and C++ compatibility

Why C

- Very widely used
  - C Compilers exist for most modern architectures and OSs
    - From super computers to embedded systems
  - Normally ranks in the top 2 of the TIOBE index
- Diverse use
  - Operating systems, device drivers, protocol stacks
  - But also for application software
- C is low level (but not assembly language)
  - The language features cleanly map to CPU features

What is C?

- "C is an imperative procedural language supporting structured programming, lexical variable scope, and recursion, with a static type system. It was designed to be compiled to provide low-level access to memory and language constructs that map efficiently to machine instructions, all with minimal runtime support. Despite its low-level capabilities, the language was designed to encourage cross-platform programming. A standards-compliant C program written with portability in mind can be compiled for a wide variety of computer platforms and operating systems with few changes to its source code."
  https://en.wikipedia.org/wiki/C_(programming_language)
- C is NOT object oriented
  - There are no classes or objects or constructors
  - It's like everything is in a single class
  - All that class stuff (private, final, etc.) doesn't exist

hello_world.c

```
#include <stdio.h>

int main(int argc, const char *argv[])
{
puts("Hello World");

return 0;
}
```

- None of that `class` stuff in C
- Programs start at `main()` which returns an `int` to the OS (0 on success, anything else on not success)

#include

- Java programs are in `.class` files, C uses `.c` and `.h`
  - `.c` for implementation
  - `.h` for `extern` declarations (Java's `public`)
- Java uses `import`, C uses `#include`

```
#include <stdio.h>
#include "myfile.h"
```

- `#include` ***literally includes the file with the given name right there into the file being compiled!***
- In `hello_world.c` we include `stdio.h` so that we can call `puts()`

Routines

- C routines look a lot like Java methods

```
int main(int argc, const char *argv[])
{
}
```

- None of that `public final` stuff

- A routine can be scoped to just the source code file

```
static int eleven(void)
```

- Routines must be declared before being called

```
extern int eleven(void);
```

If Statements

```
if (<expression>)
   <statement>
```

- For example

```
if (x == 3)
   puts("if");
```

Case Statements

```
switch (<expression>)
   {
   case <constant>:
      <statement>
   …
   default:
      <statement>
   }
```

- For example

```
switch (x)
   {
   case 0:
      puts("case Zero");
      break;
   default:
      puts("case Non zero");
   }
```

For Loops

```
for (<expression>;<expression>;<expression>)
    <statement>
```

- More usually

```
for (<init>;<cond>;<inc>)
    <statement>
```

- For example

```
for (int x = 0; x < 10; x++)
   puts("for");
```

While loops

```
while (<expression>)
    <statement>
```

- For example

```
while (x < 3)
    {
    puts("while");
    x++;
    }
```

Do Loops

```
do
   <statement>
while (<expression>);
```

- For example

```
do
   {
   puts("do");
   x++;
   }
while (x < 3);
```

Break, Continue, Return

- `break`, `continue` , `return` work like in Java

- For example

```
for (int x = 0; x < 3; x++)
   {
   if (x == 3)
      break;
   else if (x == 1)
      continue;
   else
      return;
   }
```

Constants

`const`

- A bit like `final` in Java

- For example

```
const float pi = 3.14159;
```

Enums

- Enums in C are less flexible than in Java

```
enum direction
    {
    NORTH = 0,
    SOUTH,
    EAST,
    WEST
    };
```

- Values are optional and must be integers
  - So this is just like defining a bunch of integer constants and naming a type for them

C Built-In Types

```
char,
float, double,
int, short, long,
signed, unsigned,
void
```

- For example (NOTE: ***C does NOT initialize variables***)

```
unsigned char x;
int y = 6;
```

- `void` is used for routine declarations (and pointers)
  - It means “no type” and “no parameters” here:

```
void routine(void)
```

Integer Ranges

- In C there are few guarantees of the size of a given integer type (in bits, and therefore numeric range)

Type	Size
`signed char` `unsigned char`	At least 8 bits
`signed short` `unsigned short`	At least 16 bits
`signed int` `unsigned int`	At least 16 bits
`signed long` `unsigned long`	At least 32 bits
`signed long long` `unsigned long long`	At least 64 bits

Sizeof()

- To find the size of any type (built-in or user defined) we use sizeof()

```
printf("%lu\n", sizeof(int));
```

- On my MacBook this prints 4 as an integer is 4 bytes, that is, my MacBook uses 32-bit integers.

#include <stdint.h>

- But it's often essential to know the size of an integer. For example, when reading or writing to a disk file or pipe. So we `#include <stdint.h>` and use these:

Type	Bits	Min	Max
`uint8_t`	8	0	255
`int8_t`	8	−128	127
`uint16_t`	16	0	65,535
`int16_t`	16	−32,768	32,767
`uint32_t`	32	0	4294967295
`int32_t`	32	−2,147,483,648	2,147,483,647
`uint64_t`	64	0	18,446,744,073,709,551,615
`int64_t`	64	−9,223,372,036,854,775,808	9,223,372,036,854,775,807

Structs

- Like Java classes, but without methods
  - No constructors, no setters, no getters
- All data members are public

```
struct point
   {
   float x;
   float y;
   };
```

- Yes, you can have structs in structs
- ***We'll have a whole lecture on structs later***

Typedef

- Any type can be given a new name using `typedef`

```
typedef struct
   {
   float x, y, z;
   } coordinate;
```

- Now rather than having to say `struct coordinate` we can just use `coordinate`, e.g. declare a variable

```
coordinate location;
```

Static

- The static keyword has two meanings
  - The scope of this routine is local to this source code file
  - This variable maintains its value between calls

```
void keeper(void)
{
static uint64_t times_called = 0;

times_called++;
printf("Count:%llu\n", times_called);
}
```

Typecasting

- Just like Java

```
uint64_t y = 1024;
uint32_t x = (uint32_t) y
```

Keywords To Ignore

- `auto`
  - All local variables are auto
- `register`
  - Has been deprecated
  - Hint to compiler to use a CPU register for this variable

Keywords To Think About

- `volatile`
  - Do not optimize accesses to this variable
    - Might be changed by another thread or a hardware event
  - You're likely to see this in operating system source code
  - You're likely to see this in multithreaded programs

Homework

- Read the Wikipedia article on C
  - https://en.wikipedia.org/wiki/C_(programming_language)

- If you want a good laugh:
  - The International Obfuscated C Code Contest
    - https://en.wikipedia.org/wiki/International_Obfuscated_C_Code_Contest